

Asset Reviews

Last updated: 2026-06-13. Covers the cross-org review system v2 (P0–P3), built 2026-06-12. Design source of truth: <docs/plans/cross-org-reviews.md>.

Reviews are ArtHound-native records attached to canonical assets — never synced to or from any source tool. They are the **method and record of sign-off and delivery** for an asset, serving three routines:

1. **Internal** daily craft/team review (single org, never shared).
 2. **Cross-org collaboration** — ad-hoc vendor→studio submissions and the link's required protocol submissions.
 3. **Formal delivery** — a cross-org review the studio accepts, freezing an immutable record.
-

Concepts

Scope: `internal` (default; visible only to the authoring org) or `cross_org` (lives on a studio↔vendor link; visible to both parties). Scope, link, and provenance columns are immutable after creation (trigger-enforced).

Promotion: the ONE path an internal review crosses the org wall. The `promote_review` RPC creates a **new** cross-org review (`promoted_from_review_id`) as an ACID trimmed copy — the author selects which fields, comments, and attachments cross (payload-template mental model). The internal review stays private forever. Trim selections can be saved as vendor-owned **trim templates** (`review_trim_templates`), optionally per link.

Comment lanes: every comment has `visibility ∈ internal | shared`. Internal reviews allow only the internal lane. On cross-org reviews both orgs get both lanes; the compose default is **internal everywhere** (sharing is always an explicit choice). A comment can be flipped internal→shared once (one-way, trigger-enforced, event-logged). Promotion copies selected internal comments across in the shared lane with provenance (`copied_from_comment_id`).

Multi-asset: `review_assets` is an m2m junction holding every linked asset (the primary `canonical_asset_id` stays NOT NULL on the review and is mirrored into the junction).

Link protocol (required submissions): the studio assigns ONE `review_workflow_def` to a link (`studio_vendor_links.review_protocol_def_id`); its ordered `review_step_def` rows are the required submissions per dispatched asset. Vendors fulfil a step by submitting/promoting a cross-org review tagged with `step_def_id`. The **requirements checklist** is computed at read time (`GET /api/reviews/requirements?LinkId=`): live-dispatched assets × steps; *fulfilled* = a tagged review exists; *complete* = its status is `Approved`. Tags outside the link's protocol are inert. Steps are archived, never deleted (tagged reviews keep their FK). `protocol_acknowledged_at/by` are stubs for the future explicit vendor-acceptance flow.

Acceptance: the `review_accept` RPC (studio party, active link) freezes the formal delivery: validates earlier protocol steps are fulfilled for the asset, assembles `frozen_snapshot` (review fields + studio-side asset data + **shared-lane comments only** + attachment refs by storage path, no byte copy), stamps `accepted_at/by`, sets status `Approved`. Accepted reviews are **fully immutable** — trigger blocks all updates (including RPC/system paths) and every user write policy requires an unaccepted parent.

Revision chain: re-delivery = a new review. Re-promoting the same internal review on the same link auto-links `revision_of_review_id` to the latest prior copy; ad-hoc revisions pass `revision_of_review_id` at create (validated against the same link). The prior stays frozen.

Events: `review_events` is the append-only audit trail (created / updated / status_changed / comment_added / comment_shared / promoted / attachment_added / requirement_tagged / accepted / revision_created). No user INSERT policy — routes write events under `system_identity()`; RPCs write them in-transaction.

Status: free text; current UI vocabulary `Pending / In Review / Approved / Changes Requested`. Org-definable status defs are deferred.

Security model

User-context RLS is the primary boundary (post-RLS-flip); partner actions go through `SECURITY DEFINER` RPCs owned by `arthound_rpc`.

- `asset_reviews` SELECT: authored by your org, OR `cross_org` on a link your org is party to (**any** link status — delivered records stay readable after cancellation; writes require an active link). Studios do NOT blanket-see vendor-authored reviews on their assets (the v1 `ar_sel` over-breadth was fixed in migration `20260612000002`).
- INSERT: own org as author; cross-org additionally requires an active link whose studio matches the review's studio (`is_link_party_for_studio`).
- UPDATE/DELETE: owner org only, unaccepted only; partner status transitions via the `review_set_status` RPC.
- Child tables (`review_assets`, `review_comments`, `review_events`, `review_attachments`) ride the parent review's visibility via EXISTS — the parent's RLS is the single source of truth.
- Comments: shared lane visible to any org that can see the parent; internal lane author-org only. Lane rules and the one-way flip are enforced in policy + trigger.
- Promoted attachments reference the SAME storage path (no byte copy); blob deletion first checks for other referencing rows under `system_identity()` (`copied_from_attachment_id` carries provenance).
- All review tables: FORCE RLS; registered in `scripts/rls_grant_audit.py`; persona-matrix coverage (`authored-or-linked` checks for both personas).

RPCs (all `arthound_rpc`-owned, secdef, `search_path=''`, EXECUTE for `authenticated` only):
`promote_review(uuid, uuid, jsonb, text, uuid)`, `review_set_status(uuid, text)`,

`review_set_link_protocol(uuid, uuid)`, `review_accept(uuid)`. Note: `promote_review`'s old 4-arg signature was DROPPED when `p_step_def_id` was added — PostgREST cannot dispatch overloads.

API surface

GET	/api/reviews?scope=internal cross_org all	→ list (cross_org = inbox/outbox via RLS; all = everything visible; default = aut
POST	/api/reviews	→ create; link_id ⇒ cross-org; optional step_def_id, revision_of_review_id
GET	/api/reviews/{id}	→ single (visibility-fetched) + asset cor
PATCH	/api/reviews/{id}	→ owner edits (title/description/status)
DELETE	/api/reviews/{id}	→ creator only, unaccepted only
POST	/api/reviews/{id}/promote	→ promote_review RPC {link_id, trim, step
POST	/api/reviews/{id}/status	→ review_set_status RPC (either party)
POST	/api/reviews/{id}/accept	→ review_accept RPC (studio party)
GET/POST	/api/reviews/{id}/comments	→ thread (RLS filters lanes)
PATCH/DELETE	/api/reviews/{id}/comments/{cid}	→ author edits; visibility:'shared' = lar
GET	/api/reviews/{id}/events	→ audit history
GET/POST/PATCH/DELETE	/api/reviews/trim-templates	→ vendor-owned promote defaults
GET/POST/PATCH/DELETE	/api/reviews/protocols	→ studio protocol CRUD (+ /seed-default)
POST	/api/reviews/links/{link_id}/protocol	→ assign/clear the link's protocol (RPC)
GET	/api/reviews/requirements?linkId=	→ computed checklist (both parties)
POST/GET/DELETE	/api/reviews/{id}/attachments[...]	→ upload/list/stream/delete (100 MB cap)

Asset context per side (`_enrich`): studios resolve from `replicated_assets` ; vendors resolve from their received dispatch `payload_data` . Each side sees their own view of the canonical asset. Responses carry `is_author` for UI gating.

Frontend

- **Reviews page** (`frontend/src/pages/Reviews.jsx`): Internal / Cross-org tabs; Sent/Received and Accepted pills; partner status select; Promote / Submit revision / Accept delivery actions; vendor "send to studio" + fulfilment-tag selects on create.
- **Shared components** (`frontend/src/components/reviews/`): `CommentThread` (lanes, one-way Share, readOnly when accepted), `PromoteModal` (trim checkboxes, templates, requirement tag), `ProtocolModal` (studio ordered-list editor, seed default), `RequirementsChecklist` (fixed-density table, both connections pages).
- **Connections pages**: the studio's `/vendors` page (`VendorConnections.jsx`) hosts the protocol editor + checklist + real Open Reviews; the vendor's `/studios` page (`StudioConnections.jsx`) shows a Review-protocol pill + expandable requirements per link. (Page names refer to the counterparty.)
- **Asset Viewer** Reviews tab fetches `scope=all` so partner-shared reviews appear on the asset.

Relationship to the Handshake

The link's `review_collaboration_mode` (`none` / `isolated` / `collaborative`) is **vestigial**: ad hoc cross-org reviews are always available on an active link (requirement), and the real cross-org review configuration is the link's protocol (`review_protocol_def_id`). The column is kept but unread.

Deferred

Per-record grants (`review_grant` ; the schema is grant-compatible), org-definable statuses (`review_status_def`), explicit vendor protocol acceptance (stub columns shipped), vendor-owned protocols, stages/actors on step defs, studio↔studio reviews, notifications (→ notification system TODO), cadence scheduling, per-studio review field visibility config (pre-existing TODO).